

TIRA: Toolbox for Interval Reachability Analysis

User manual, version 2

Pierre-Jean Meyer, Alex Devonport, Neelay Junnarkar, Murat Arcak

2021-12-03

1 Overview

TIRA is a Matlab library gathering into a unified framework several methods for the computation of interval over-approximations of the finite time reachable set of a continuous-time or discrete-time system described by

$$\begin{aligned}\dot{x} &= f(t, x, p), \\ x^+ &= F(t, x, p),\end{aligned}$$

with time $t \in \mathbb{R}$, state $x \in \mathbb{R}^{n_x}$ and input $p \in \mathbb{R}^{n_p}$.

This document first provides

- the installation instructions (Section 2),
- the toolbox architecture (Section 3),
- and an overview of the methods currently implemented in TIRA and their requirements (Section 4).

Then, it gives the basic instructions for the main uses of TIRA:

- Test the implemented reachability methods on the pre-defined examples (Section 5).
- Define your own system and reachability problem, and call one of the implemented reachability methods on it (Section 6).
- Define your own reachability method and add it to the toolbox (Section 7).

2 Installation and requirements

2.1 Requirements

The tool TIRA is implemented in Matlab. It has been tested on Matlab version R2021a, but should be compatible with versions R2017b or superior.

Toolbox dependencies. TIRA can be successfully run with the base Matlab without additional toolbox. However, we should highlight the following points:

- Function `Utilities\Sensitivity_falsification.m` (in method 4 *Sampled-data mixed-monotonicity*, submethod 0 *Sampling and falsification*) calls function `fmincon` which requires the *Optimization Toolbox* (`optimization_toolbox`). TIRA automatically checks the availability of this toolbox and skips the falsification function if it is unavailable.
- Function `Utilities\Monotonicity_sign_conditions.m` (in method 1 *Continuous-time monotonicity*) calls function `glineq` when the *Communications Toolbox* (`communication_toolbox`) is available, and otherwise calls an alternative method implemented by the authors that does not require this toolbox. Note that the Matlab dependency tools also report the use of the *DSP System Toolbox* (`signal_blocks`) when calling function `glineq`, although this toolbox should in theory not be needed.

2.2 Installation instructions

TIRA can be downloaded from the public repository available at https://gitlab.com/pj_meyer/TIRA. To use it,

- either set Matlab's *current folder* to the main TIRA folder and work directly in this folder,
- or work from a remote folder by adding the downloaded TIRA folder and all its subfolders to the Matlab search path. For this, you can use the command `addpath(genpath('full_path_to_TIRA_folder'))`

3 Tool architecture

The overall architecture of the main TIRA folder and the toolbox is described below.

- `System_description.m` is where the user defines their own system (either continuous-time or discrete-time)
- `MAIN_CALL.m` is where the user defines the reachability problem to be solved (intervals of initial states and inputs, time range) and calls the hub function `OA_methods/TIRA.m`
- `Solver_parameters.m` allows the user to choose which reachability method to apply (if none is chosen, TIRA will automatically pick the most suitable one) and modify the internal solver parameters.
- folder `Input_files/` contains skeleton files that can be filled by the user to give further useful information about the considered system (e.g. information about the Jacobian matrices). The list of possible user-provided system information (and the corresponding reachability methods they enable) is given in Section 4.3.

- folder `OA_methods/` contains the hub function `OA_methods/TIRA.m` as well as one function for each of the 7 reachability methods currently implemented in TIRA. Users do not need to modify anything in this folder.
 - `OA_methods/TIRA.m` receives as input the definition of the reachability problem. If a specific reachability method is requested (as input argument to the function, or by modifying `Solver_parameters.m`), this function checks if the requirements for this methods are satisfied by the system and the user-provided informations in folder `Input_files/`, and then calls the corresponding over-approximation method (or returns an error if the requirements are not satisfied). If no specific method is requested, this function iteratively checks the requirements of each over-approximation method, and calls the first one whose requirements are satisfied.
 - `OA_methods/OA_...` are the files for each reachability method. An overview of these methods and their requirements is provided in both file `Solver_parameters.m` and Section 4.2 of this document.
- folder `Utilities/` contains various functions that are called by the hub function `OA_methods/TIRA.m` or one of the reachability methods. Users do not need to modify anything in this folder.
- folder `Examples/` contains a copy of some of the main files (`System_description.m`, `MAIN_CALL.m`, `Solver_parameters.m` and all files from folder `Input_files/`) already filled with the reachability problems for 13 examples. See Section 5 to use this folder.
- folder `Test_suite/` also contains a copy of these pre-filled files as well as the script `Run_tests.m`. When running this script, it calls all currently implemented reachability methods and submethods on the pre-defined examples and scans their outputs. This file can be used to ensure that future changes in the toolbox did not break anything on the existing files and methods.

The toolbox architecture (as of version 1 of TIRA) is also sketched in Figure 1 highlighting

- in the center the hub function `TIRA.m` calling each reachability method, and that can be easily extended with additional methods,
- in red the required inputs of the system description in `System_description.m` and the reachability problem definition in `MAIN_CALL.m`,
- in purple the optional but recommended additional user-provided inputs in folder `Input_files/` (detailed in Section 4.3),
- and in blue the more optional inputs in `Solver_parameters.m`, such as the choice of the reachability method, or changing the default values of the solvers' internal parameters.

All Matlab files in TIRA are heavily commented to ensure that users can fully understand their behaviors. In particular, the top of each function contains the following information:

- Plain-word description of what the function does.
- References to publications in which the theory behind these functions can be found.
- List of input arguments of the function.
- List of outputs that the function returns.

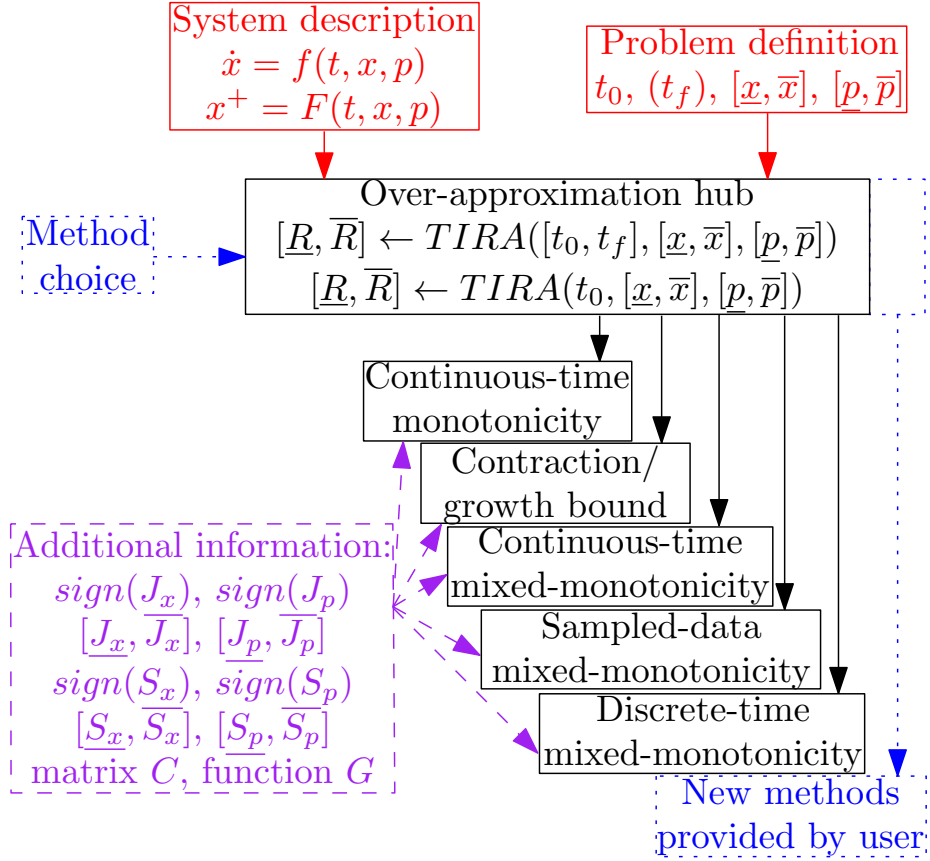


Figure 1: Sketch of the toolbox architecture.

4 Reachability method descriptions

4.1 References

- The toolbox architecture and a brief description of the first 5 reachability methods implemented in version 1 of TIRA (2019-02-19) are presented in the HSCC'19 tool paper [1] available in folder Documentation of the GitLab repository.

- A much more thorough presentation of all 7 reachability methods implemented in version 2 of TIRA (2021-12-03) and their submethods is provided in the book [2]. The tutorial-like formulation of this book presents a unified framework, where for each reachability method we clearly identify the requirements and limitations, the main steps for the reachability analysis, and the final reachable set over-approximation. A partial preprint of this book (containing the preface, table of content, and Appendix B related to TIRA) is available in folder Documentation of the GitLab repository. If you are interested in this book, feel free to contact the authors directly.

4.2 Methods and requirements

Each method and function in the toolbox is heavily described in the comments of the corresponding Matlab files. In particular, file `Solver_parameters.m` contains a list of all implemented reachability methods and their requirements, which we also summarize below.

1. **Continuous-time monotonicity.** This choice will check if the system is monotone, then apply an over-approximation method specific to monotone systems. Pick option 3 for a slightly slower version but which does not require checking the monotonicity satisfaction, yet will still provide the same results if the system is indeed monotone.

Requirements:

- Continuous-time system
- One of the following 2 items:
 - Signs of the Jacobian matrices in `Input_files/UP_Jacobian_Signs.m`
 - Bounds of the Jacobian matrices in `Input_files/UP_Jacobian_Bounds.m`

2. **Contraction and growth bound.** This method relies on using or defining a function that bounds the growth or contraction on each dimension of the system.

Requirements:

- Continuous-time system
- One of the following 3 items:
 - Growth bound function in `Input_files/UP_Growth_Bound_Function.m`
 - Contraction matrix or scalar in `Input_files/UP_Contraction_Matrix.m`
 - Bounds of the state Jacobian in `Input_files/UP_Jacobian_Bounds.m`
 The last 2 cases also require dynamics with additive input:

$$\text{System_description}(t, x, p) = \text{System_description}(t, x, 0) + p$$

which is automatically checked in `Utilities/Growth_bound_choice.m`

3. **Continuous-time mixed-monotonicity.** This method is an extension of method 1 to any system with bounded Jacobian matrices.

Requirements:

- Continuous-time system

- One of the following 2 items:
 - Signs of the Jacobian matrices in `Input_files/UP_Jacobian_Signs.m`
 - Bounds of the Jacobian matrices in `Input_files/UP_Jacobian_Bounds.m`

4. **Sampled-data mixed-monotonicity.** This method applies the discrete-time mixed-monotonicity method (5) to the sampled-data version of a continuous-time system. The requirements on the Jacobian of the discrete-time system thus translate into requirements on the sensitivity matrices of the continuous-time system.

Requirements:

- Continuous-time system
- Input p constant over the considered time range $[t_{\text{init}}, t_{\text{final}}]$.
- Optional user-provided inputs amongst the following items:
 - Signs of the sensitivity matrices in `Input_files/UP_Sensitivity_Signs.m`
 - Bounds of the sensitivity matrices in `Input_files/UP_Sensitivity_Bounds.m`
 - Bounds of the Jacobian matrices in `Input_files/UP_Jacobian_Bounds.m`
 - Bounds of the 2nd order Jacobian matrices in `Input_files/UP_Jacobian_2ndOrder_Bounds.m`
 - Jacobian functions in `Input_files/UP_Jacobian_Function.m`
 If none of these optional input is provided, the method can still be called using default submethod 4-0 *Sampling and falsification*.

5. **Discrete-time mixed-monotonicity.** Similar to method 3 but for discrete-time systems.

Requirements:

- Discrete-time system
- One of the following 2 items:
 - Signs of the Jacobian matrices in `Input_files/UP_Jacobian_Signs.m`
 - Bounds of the Jacobian matrices in `Input_files/UP_Jacobian_Bounds.m`

6. **Quasi-Monte Carlo.** This method simulates the system over a grid of points and combines it with an error-bound function to over-approximate the reachable set.

Requirements for discrete-time systems:

- Error-bound function in `Input_files/UP_Error_Bound_Function.m`

Requirements for continuous-time systems: one of the following 4 items

- Error-bound function in `Input_files/UP_Error_Bound_Function.m`
 - Lipschitz constant in `Input_files/UP_Lipschitz_Constant.m`
 - Contraction matrix or scalar in `Input_files/UP_Contraction_Matrix.m`
 - Bounds of the Jacobian matrices in `Input_files/UP_Jacobian_Bounds.m`
- The last 2 cases also require dynamics with additive input:

$$\text{System_description}(t, x, p) = \text{System_description}(t, x, 0) + p$$

which is automatically checked in `Utilities/Error_bound_choice.m`

7. **Monte Carlo.** This method simulates the system over uniformly sampled points to generate an approximation interval with a desired probabilistic accuracy and confidence. The resulting interval approximation is not guaranteed to be an OVER-approximation of the reachable set. This method has no requirement and is applicable to both discrete-time and continuous-time systems.

4.3 User-provided system information

While a couple of methods can be run without any additional requirement than defining the considered system and reachability problem (such as method 4 *Sampled-data mixed-monotonicity* with submethod 0 *Sampling and falsification*; or method 7 *Monte Carlo*), most of them rely on additional user-provided information on the system. Below, we list all possible user-provided input files and the reachability methods that they enable. Mathematical details on the required format of these information are provided in the corresponding Matlab files.

- **Input_files/UP_Contraction_Matrix.m:** contraction matrix or contraction scalar on the dynamics of a continuous-time system.
 - Method 2 *Contraction and growth bound*
 - Method 6 *Quasi-Monte Carlo*
- **Input_files/UP_Error_Bound_Function.m:** handle of an error bound function for either continuous-time or discrete-time system.
 - Method 6 *Quasi-Monte Carlo*
- **Input_files/UP_Growth_Bound_Function.m:** handle of a growth bound function for a continuous-time system.
 - Method 2 *Contraction and growth bound*
- **Input_files/UP_Jacobian_2ndOrder_Bounds.m:** bounds on all 4 second-order Jacobian matrices (with respect to initial states and inputs) of a continuous-time system.
 - Method 4 *Sampled-data mixed-monotonicity* with submethod 3 *Second-order sensitivity*
- **Input_files/UP_Jacobian_Bounds.m:** bounds on both Jacobian matrices (with respect to initial states and inputs) for either continuous-time or discrete-time system.
 - Method 1 *Continuous-time monotonicity*
 - Method 2 *Contraction and growth bound*
 - Method 3 *Continuous-time mixed-monotonicity*
 - Method 4 *Sampled-data mixed-monotonicity* with submethod 2 *Interval arithmetics*
 - Method 5 *Discrete-time mixed-monotonicity*

- Method 6 *Quasi-Monte Carlo*
- **Input_files/UP_Jacobian_Function.m**: function handle of both Jacobian matrices (with respect to initial states and inputs) for a continuous-time system.
 - Method 4 *Sampled-data mixed-monotonicity* with submethods 0 *Sampling and falsification* or 3 *Second-order sensitivity*, when using the ODE solver 2 for the sensitivity equations.
- **Input_files/UP_Jacobian_Signs.m**: signs on both Jacobian matrices (with respect to initial states and inputs) for either continuous-time or discrete-time system, if the system has sign-stable Jacobian matrices over the considered sets of states and inputs.
 - Method 1 *Continuous-time monotonicity*
 - Method 3 *Continuous-time mixed-monotonicity*
 - Method 5 *Discrete-time mixed-monotonicity*
- **Input_files/UP_Lipschitz_Constant.m**: Lipschitz constant on the dynamics of a continuous-time system.
 - Method 6 *Quasi-Monte Carlo*
- **Input_files/UP_Sensitivity_Bounds.m**: bounds on both sensitivity matrices (with respect to initial states and inputs) of a continuous-time system.
 - Method 4 *Sampled-data mixed-monotonicity* with submethod 0 *Reading sensitivity information directly from user-provided files*
- **Input_files/UP_Sensitivity_Signs.m**: signs on both sensitivity matrices (with respect to initial states and inputs) of a continuous-time system, if the system has sign-stable sensitivity matrices over the considered sets of states and inputs.
 - Method 4 *Sampled-data mixed-monotonicity* with submethod 0 *Reading sensitivity information directly from user-provided files*

5 Using TIRA on the pre-defined examples

To run one of the implemented reachability methods on one of the provided example systems, follow the steps below.

- Set the **Examples** folder as Matlab's new *current folder*. (All files listed below are those in this **Examples** subfolder.)
- Open function **System_description.m** which lists the system descriptions of the examples currently implemented in TIRA. Look at the **switch/case** number on variable **system_choice** corresponding to the system you want to use.

- Open file `MAIN_CALL.m` and in section “Choice of the example system” (top of the file), uncomment the line `system_choice = ...` corresponding to your system (and comment the others).
- In section “Definition of the reachability problem (for each example system)”, look up the `switch/case` number of your system and modify the definition of the reachability problem (`x_low`, `x_up` for the interval of initial states, `p_low`, `p_up` for the interval of inputs, `t_init`, `t_final` for the time range of the reachability problem) as desired (or use the provided default values).
- Then, two choices are available.
 - Either run the script `MAIN_CALL.m` directly. This will let TIRA go through each available methods until it finds one whose requirements are satisfied, then apply it. It will not look for other methods after one is found.
 - If you want to request TIRA to apply a specific method, open file `Solver_parameters.m`, and replace `parameters.OA_method = NaN;` by the number of the method you want to use (the file details which method corresponds to each number). This file can also be used to request a specific submethod or change the internal solver parameters (details are provided in the comments of the file). Then run the script `MAIN_CALL.m`. Since in this case you requested a specific method, TIRA will return an error if the chosen system does not satisfy the requirements for the chosen method.

This call will display in the Matlab console the method(s) tried by TIRA and their running times. If a method is successfully applied, it will also plot the resulting interval alongside an estimation of the actual reachable set (obtained from computing the successors of 1000 random initial states). This plotting is only enabled for systems whose state dimension is between 2 and 20.

If instead of the file `MAIN_CALL.m`, you follow the same procedure using file `MAIN_CALL_OA_comparison.m`, then all main over-approximation methods will be tried at once on the same problem. The methods which cannot be applied to the considered system and reachability problem are ignored, and the resulting over-approximations of all other methods are plotted in the same graph.

6 General use of TIRA on user-provided system

A more general use of the tool TIRA involves the user providing their own system description and problem definition. While the use of TIRA in such case is similar to what is described in Section 5 above, there are some differences highlighted below.

- Set the main TIRA folder as Matlab’s new *current folder*. (All files listed below are those in this TIRA folder.)
- Open function `System_description.m` and provide your own system description (this file can be used for both continuous-time and discrete-time systems).

- Provide additional information on the system in one or more files in subfolder `Input_files` (e.g. known bounds for the Jacobian matrices, contraction matrix, Lipschitz constant). This step is optional but recommended since most methods implemented in TIRA rely on at least one of these user-provided information. The list of requirements for each methods implemented in TIRA is provided in `Solver_parameters.m`. If the user does not know which information to provide or which reachability method to use, we advise to provide upper and lower bounds on the Jacobian matrices (in file `Input_files/UP_Jacobian_Bounds.m`) which is the most versatile and can be exploited in most reachability methods (although some of the methods have additional requirements that will be automatically checked for satisfaction by TIRA).
- Open file `MAIN_CALL.m` and define the reachability problem that you want to solve (`x_low`, `x_up` for the interval of initial states, `p_low`, `p_up` for the interval of inputs, `t_init`, `t_final` for the time range of the reachability problem). `t_final` is only needed for continuous-time systems.
- In `MAIN_CALL.m`, section “Call of the main over-approximation function” (shown below), uncomment the call of the TIRA function corresponding to the type of system you defined (continuous-time or discrete-time).

```
% Call for continuous-time systems over the time range [t_init
,t_final]
% [succ_low,succ_up] = TIRA([t_init,t_final],x_low,x_up,p_low,
p_up);

% Call for discrete-time systems for one step starting from
time t_init
% [succ_low,succ_up] = TIRA(t_init,x_low,x_up,p_low,p_up);
```

- Then, the same two choices as in Section 5 are available: running the script `MAIN_CALL.m` directly to let the tool pick a method; or request a specific method by modifying the file `Solver_parameters.m`.

This call will display in the Matlab console the method(s) tried by TIRA and their running times. To obtain plots as in Section 5, copy the last two sections of `Examples/MAIN_CALL.m` to the end of `MAIN_CALL.m` (you also need to define beforehand variable `bool_discrete_time` to 0 if you have a continuous-time system or to 1 for a discrete-time system).

A short illustrated tutorial of the above steps is also provided in Appendix B of the authors’ book [2] available in folder Documentation of the GitLab repository.

7 Adding your own reachability methods

The architecture of TIRA sketched in Section 3 was chosen to allow users to easily extend it with additional interval-based reachability methods. For this, we suggest to follow the procedure below.

- Define the main function of your over-approximation method in a separate file to be added in the folder `OA_methods/`. If this method needs to call any side functions, you can add them in folder `Utilities/`.

- In `Solver_parameters.m`, give a number to your method, and optionally add the default values for its internal parameters.
- In the hub function `OA_methods/TIRA.m`, add the call to this function taking inspiration from the format of the existing methods. If this new reachability method has some requirements, make sure that all these requirements are checked within `OA_methods/TIRA.m` before the call of your function.

References

- [1] P.-J. Meyer, A. Devonport, and M. Arcak. TIRA: Toolbox for interval reachability analysis. In *22nd ACM International Conference on Hybrid Systems: Computation and Control*, pages 224–229, 2019.
- [2] P.-J. Meyer, A. Devonport, and M. Arcak. *Interval Reachability Analysis: Bounding Trajectories of Uncertain Systems with Boxes for Control and Verification*. Springer, 2021.